

Large Language Models as Decision-Makers Under Uncertainty: Value of Information, Error Decomposition and an Extract-then-Solve Guardrail

Chiheb Mejrissi*

Department of Computer Science, Example University, 1 Example Street, Example City, Country

Abstract

[Intent: Problem (decisions under uncertainty handed to LLMs); the benchmark with exact Bayes-optimal actions on open-weight models; the two headline questions, value-of-information pricing (C5) and where the loss arises (C6); the guardrail (C4, C4b); the replications (C1, C2(b)); outcomes once recorded.]

Keywords: large language models, decision support, value of information, Bayesian decision theory, calibration, guardrails, benchmark

1. Introduction

Large language models (LLMs) are increasingly asked to advise on decisions: whether to order a confirmatory test before treating a patient, whether to escalate an alert, how much stock to order against uncertain demand, whether a piece of evidence should change a course of action. Such questions arrive as text, and a model answers them in text, so an LLM can be placed in front of a decision problem with no modelling effort by its user. That convenience is also the risk. A decision under uncertainty has a right answer only relative to probabilities and costs, and a fluent recommendation that ignores a base rate, a cost asymmetry or the price of a test can look as reasonable as the Bayes-optimal one. For a decision support system, the relevant question

*Corresponding author.

Email address: `chiheb.mejrissi@example.org` (Chiheb Mejrissi)

is therefore not whether a model’s answers sound sensible, but how much expected loss they carry compared with the best available action, where that loss comes from, and which part of the decision should be taken away from the model.

Behavioural studies have established that LLMs reproduce some human decision heuristics and biases and avoid others, depending on the model and its version [1, 2, 3, 4, 5], that their stated probabilities fall short of Bayesian updating [6], and that their decisions can change with the wording of a question [7]. The closest recent work scores LLM decisions against an exact optimum on binary tasks and splits their regret into a belief loss and an optimization residual [TODO: verify N1], or tests whether a model’s stated probabilities are coherent with its own choices [TODO: cite: Yamin et al. 2026 (L28), ICLR; bib entry pending a venues.csv row]. Section 2 reviews these strands. Three questions that matter for decision support remain open. First, whether a model pays for information exactly when the information is worth its cost; this is the value-of-information decision of classical decision analysis [8], and no study we found scores it against its exact optimum. Second, when a direct decision loses value, whether the loss comes from misreading the problem, from inferring a wrong probability from a correct reading, or from failing to act on the model’s own probability; the answer determines what a guardrail must protect. Third, whether handing the computation to an exact solver, with the model reduced to extracting the problem’s quantities, removes the loss, keeps the decision invariant under rewordings that should not change it, and does better than letting the model write the solver itself.

This paper answers these questions with a benchmark built for the purpose (Section 3). Its items are decision problems stated in text, drawn from four generated families: verification and stopping, asymmetric-cost triage, allocation under uncertain demand, and Bayesian updating. For every item an exact solver computes the Bayes-optimal action and the expected loss of every action, so every answer is scored by its normalized regret against the optimum, with no human or model judge. Because the items are generated from registered seeds, they cannot have appeared in any model’s training data; because every expected loss is known, an item can be restated in ways that provably leave the decision unchanged. Open-weight models with pinned weights are evaluated under a fixed set of decision conditions (Section 4), which range from answering directly, through reasoning, structured answers, model-written code and a stated decision rule, to an extract-then-

solve guardrail in which the model only extracts the item’s quantities into a fixed schema and the exact solver chooses the action. Every metric, test and reading rule was fixed before any model was run (Section 5).

The study is organized around two headline questions, one design question and a set of replications.

- *Do direct decisions misprice information?* (C5.) In the verification family, buying the test is optimal on half of the items and a wrong action on the other half. The value-of-information discrimination of a model is the difference between its rate of buying the test when buying is optimal and when it is not; the exact solver attains the maximum. The benchmark tests whether this discrimination is low when models decide directly and high under the guardrail.
- *Is the loss misreading, inference or action?* (C6.) In the structured-direct condition the model states its reading of the item’s quantities, its probability and its action in one answer. The regret of the action then splits exactly into a reading term, an inference term and an action term, and the benchmark tests whether most of the loss is not misreading.
- *What does a verified solver hand-off buy?* (C4, C4b.) The benchmark tests whether the extract-then-solve guardrail removes most of the decision error, so that what remains is extraction error, and whether it does better than program-of-thought, in which the model writes the solver as code [9], in regret and in invariance under reframing.
- *Replications.* On multi-action families and pinned open weights, the benchmark tests whether direct decisions are not Bayes-optimal and lose where a named heuristic misleads (C1), with wrong answers following that heuristic (C1’); whether stated probabilities are neither Bayesian nor coherent with the model’s own choices (C2); and whether decisions move under changes that should not move them (C3). C1 and the coherence part of C2 repeat the closest prior designs on a new object and are presented as replications.

The paper makes four contributions. (i) A benchmark of decision problems with exact Bayes-optimal actions, spanning costly information, cost-sensitive thresholds, allocation and updating, with trap strata built around named heuristics, decision-equivalent variants checked numerically, a measured noise floor and contamination control by generation; the generators,

solvers and harness are released. (ii) A measure of value-of-information pricing scored against the exact optimum, which tests whether a model buys information exactly when it is worth its cost. (iii) A three-stage decomposition of decision regret into reading, inference and action, which uses the model’s own extraction as a measurement instrument and extends the two-stage belief and optimization split by the stage at which a guardrail can fail. (iv) A pre-specified comparison of an extract-then-solve guardrail with direct answers, free reasoning and program-of-thought on the same items, against the exact optimum and under reframing, which turns the evaluation into a design recommendation for decision support: what to leave to the model and what to hand to a solver. The claims are read on the tested panel only and are stated with their reading rules in Section 5; negative outcomes are reported beside positive ones.

Section 2 positions the paper against LLM decision benchmarks, Bayesian decision tasks and extract-then-solve pipelines. Section 3 describes the benchmark, Section 4 the models and decision conditions, and Section 5 the pre-specified analysis. Section 6 reports the outcomes, Section 7 discusses what they imply for the design of decision support and where they stop, and Section 8 concludes.

2. Related work

Three bodies of work meet in this paper: studies and benchmarks of LLMs as decision-makers, the decision-theoretic tasks on which Bayesian behaviour can be scored exactly, and pipelines in which a model extracts a problem and a formal engine solves it. For each, this section states what is established and what the benchmark adds.

2.1. *LLMs as decision-makers*

A first line of work transfers the vignettes of cognitive psychology to LLMs. Binz and Schulz [1] submitted a language model to decision-making, information-search and reasoning tasks taken from that literature. Hagen-dorff et al. [2] report that human-like intuitive behaviour and reasoning biases emerged in large language models but disappeared in a later chat model, so a finding about one model version need not carry to the next. Macmillan-Scott and Musolesi [3] evaluate models on classic tasks from the cognitive-bias literature, and Suri et al. [4] ask whether a model shows the decision heuristics documented in humans. Closer to decision support, Chen et al. [5] run a

battery of operations-management biases on a chat model and find that it mirrors many human biases, performing better than people on some probability tasks and worse on preference-based ones. Cheung et al. [7] find that models advising on moral dilemmas show an amplified omission bias and a bias towards answering no, so that their advice flips with the wording of the question. Chen et al. [10] offer a counterpoint: on budget-allocation tasks, the choices of a model are largely consistent with utility maximization. Consistency with the axioms of rational choice, however, is not optimality against known probabilities and costs, and it is the latter that a decision support system needs.

Several studies evaluate LLM decisions under uncertainty specifically. Jia et al. [TODO: cite: Jia et al. 2024 (L7), NeurIPS; bib entry pending a venues.csv row] elicit risk preferences, probability weighting and loss aversion from multiple-choice lists; these are preference parameters with no correct value. DeLLMa [TODO: cite: Liu et al. 2025 (L23), ICLR; bib entry pending a venues.csv row] scaffolds a decision by having the model forecast the unknown states, elicit a utility and maximize expected utility, and scores the result on realized outcomes rather than against an exact optimum; the model still performs every step. In inventory problems, models have been reported to show pull-to-centre and demand chasing, which persist when the optimal formula is stated [TODO: cite: Liu, Chen & Zhong 2025 (L32), preprint; sole-source exception pending], and framing effects [TODO: cite: Zhao et al. 2025 (L33), preprint; sole-source exception pending].

Two recent studies share this paper’s object most closely. Sun et al. [TODO: verify N1] generate binary decision tasks with exact posteriors and stated costs, measure the regret of model decisions against the Bayes action, and split it into a belief loss and an optimization residual, the gap between the model’s action and the Bayes action at its own stated belief. Yamin et al. [TODO: cite: Yamin et al. 2026 (L28), ICLR; bib entry pending a venues.csv row] derive tests of whether elicited probabilities and actions are coherent without assuming a utility, and find violations on medical diagnosis tasks. Regret against an exact optimum (C1) and belief–action incoherence (C2) are therefore established, and this paper repeats them as replications on another object. It differs from both studies in four respects that correspond to its contributions: its families have more than two actions and include a costly-information option, sequential tests and allocation; it scores the purchase of information against its exact value; it adds a reading stage to the decomposition of regret; and it compares the model deciding with the model

extracting for an exact solver and with the model writing code, both against the optimum and under decision-equivalent rewordings.

2.2. Bayesian decision tasks and stated probabilities

The benchmark’s families are textbook decision problems with closed-form or exactly computable solutions. The value of a test is the reduction in expected loss that its result allows, and the test is worth buying exactly when that value exceeds its cost [8]. The treatment threshold sets the posterior above which intervention is optimal from the ratio of the costs of false positives and false negatives [11]. The newsvendor’s critical fractile and Bayes’ rule for updating complete the set. Human decision-makers are known to depart from these solutions through heuristics such as base-rate neglect and framing effects [TODO: cite: Tversky & Kahneman 1981 (L17), Science; original-source exception pending], and each family’s trap stratum is built around one such heuristic (Section 3.2).

For LLMs, stated probabilities have been found to fall well short of Bayesian updating, a shortfall that training on a Bayesian model’s predictions reduces [6], and in-context evidence has been reported to move models towards Bayesian behaviour [TODO: cite: Gupta et al. 2025 (L36), ACL; bib entry pending a venues.csv row]. Base-rate neglect appears among the biases reported for classic tasks [3]. Calibration of the confidence that a model conveys is a distinct concern: users tend to overestimate the accuracy of model answers, and explanations that reflect the model’s internal confidence narrow that gap [12]. Whether a model acts on its own stated probability has been studied through abstention under stated error penalties [TODO: cite: Wang et al. 2026 (L29), preprint; sole-source exception pending] and through the coherence tests above. On information acquisition, making a prior explicit to an agent has been reported to improve its decisions on when to stop exploring [TODO: cite: Ding, Tomlin & Durrett 2026 (L38), preprint; sole-source exception pending]. None of these studies scores a decision to buy a test against the test’s exact value of information with stated test costs, which is what the value-of-information discrimination of Section 5.1 measures.

2.3. Extract-then-solve pipelines

Delegating computation from a language model to a formal engine is an established pattern [13]. Program-of-thought prompting has the model write a program for a numerical question and executes it, separating the computation from the reasoning [9]; program-aided language models follow the same

idea [TODO: cite: Gao et al. 2023, PAL (L26), ICML; metadata unverified; bib entry pending a venues.csv row]. For logical reasoning, a model can translate a problem into a symbolic formulation that a solver then resolves [TODO: cite: Pan et al. 2023, Logic-LM (L12), Findings of EMNLP; bib entry pending a venues.csv row], and for probability questions stated in text, a model can emit a probabilistic program that is evaluated exactly [TODO: cite: Nafar, Venable & Kordjamshidi 2025, BLInD (L37), AAAI; bib entry pending a venues.csv row]. In operations research, models are trained or prompted to formulate optimization problems from natural-language descriptions for a solver [14] [TODO: cite: AhmadiTeshnizi, Gao & Udell 2024, OptiMUS (L39), ICML; metadata unverified; bib entry pending a venues.csv row]. In decision analysis, a model has been used to build an influence diagram from text [TODO: cite: Hong, Qin & Wang 2026, LAMDA, Decision Analysis (Q3 SJR 2025); exception needed], and in inventory control, methods in which a model supplies or adjusts the parameters of an operations-research policy have been compared with the model alone and the policy alone [TODO: cite: Baek et al. 2026 (L34), preprint; sole-source exception pending].

These pipelines are usually scored on answer accuracy or on the quality of the formulation. The guardrail of this paper is the same pattern applied to a decision, but evaluated differently in three respects. It is scored by the regret of the resulting action against the exact optimum, so an extraction error counts in proportion to the loss it causes. It is tested for invariance under decision-equivalent rewordings, which a robust hand-off should leave unchanged. And it is compared on the same items with program-of-thought, in which the model writes the model of the problem as well as its solution, so that the value of a *verified* solver can be separated from the value of computing in code (C4b). The model’s own extraction also serves as a measurement instrument: in the structured-direct condition it locates the stage at which a direct decision loses value (C6).

3. Benchmark: decision problems with exact optimal actions

The benchmark consists of decision problems stated in text whose Bayes-optimal action is computed exactly by a solver. Exactness is the design principle: it lets every answer be scored by its expected loss against the optimum, with no human or model judge, and it lets the same item be restated in ways that provably leave every expected loss unchanged. The

generators, solvers, strata, variants and checks described in this section were fixed before any model was run.

3.1. Task families and exact solvers

Each item presents a decision-maker with two to several named actions, the probabilities and costs needed to evaluate them, and a short story that gives the problem a setting. Four families cover the decisions the paper is about: whether to pay for information before acting, where to set an action threshold when errors cost different amounts, how much to commit against uncertain demand, and how to update a belief from evidence. Each family has an exact solver and a named *trap heuristic*, a plausible shortcut that some items are built to make wrong (Section 3.2).

F1: verification and stopping. Two causes H_1 and H_0 are possible, with prior $p = \Pr(H_1)$. The decision-maker may act for H_1 , act for H_0 , or first buy a test of cost c with sensitivity s and specificity t and then act on its result. Acting for H_0 when H_1 holds costs a miss M ; acting for H_1 when H_0 holds costs a false alarm A . In sub-family F1a ([TODO: param: F1a items] items) a single test is available and the solver compares the expected loss of acting now with that of testing and then acting optimally on the result. In F1b ([TODO: param: F1b items] items) up to $T \in$ [TODO: param: F1b test counts] conditionally independent tests can be bought in sequence, each at cost c , and the solver computes the optimal policy by backward induction. The optimal first action is to test exactly when the value of the test’s information exceeds its cost [8]. F1 is therefore the family on which information pricing is measured (Section 5.1). Trap heuristic: act for the more probable cause, ignoring the costs and the test.

F2: asymmetric-cost triage. A case has prior p of being positive and one test result with sensitivity s and specificity t . A missed positive costs C_{FN} and a needless intervention costs C_{FP} . The Bayes action is to intervene if and only if the posterior exceeds $C_{FP}/(C_{FP} + C_{FN})$, the classical treatment threshold [11]. Trap heuristic: intervene if the posterior exceeds one half, ignoring the cost asymmetry.

F3: allocation under uncertain demand. In F3a ([TODO: param: F3a items] items), a newsvendor faces a discrete demand on [TODO: param: F3a support points] support points with stated probabilities, a unit price, a unit cost and a salvage value, and chooses an order quantity among the support

points. The solver returns the smallest quantity whose cumulative probability reaches the critical fractile. In F3b ([TODO: param: F3b items] items), a fixed stock is split between two sites with independent discrete demands, and the solver enumerates the splits. Trap heuristic: order, or split by, the mean demand rounded to the nearest support point. F3 asks for no probability, only an action.

F4: Bayesian updating. A hypothesis H_1 has prior p , and [TODO: param: F4 evidence count] conditionally independent test results or urn draws with stated likelihoods are observed. The decision-maker states the posterior and then accepts or declines a bet that pays X if H_1 holds and loses Y otherwise; the Bayes action accepts if and only if the exact posterior p^* satisfies $p^*X > (1 - p^*)Y$. Trap heuristic: base-rate neglect, the posterior computed from the likelihood ratio alone.

3.2. Regret, strata and decision-equivalent variants

Normalized regret. Let $L(a)$ be the exact expected loss of action a on an item, $L^* = \min_a L(a)$ and $L_{\max} = \max_a L(a)$ over the item's actions. The normalized regret of a is

$$r(a) = \frac{L(a) - L^*}{L_{\max} - L^*} \in [0, 1], \quad (1)$$

which is zero for the Bayes action and one for the worst listed action, so items with different stakes and currencies are comparable. A model is *practically optimal* in a family if its mean normalized regret is below $\delta =$ [TODO: param: delta], a margin fixed before any run.

Separation. An item is kept only if the second-best action's expected loss exceeds the best by at least [TODO: param: separation margin] ($L_{\max} - L^*$). Near-ties, on which a small numerical slip would change the optimal action at almost no cost, are thereby excluded, and every wrong action carries a regret bounded away from zero.

Trap and non-trap strata. An item is a *trap* item if its family's heuristic gives a wrong action on it and a *non-trap* item if the heuristic gives the Bayes action. Each family is drawn in equal halves from the two strata by rejection sampling on the heuristic. The strata are defined by construction (check K1, Section 4.3): each heuristic has positive regret on every trap item and zero

regret on every non-trap item. A model that follows a heuristic therefore loses exactly on the trap stratum, and the difference between strata locates the loss.

Balance of the optimal action. Two families are further balanced on their Bayes action, because a family in which one action is nearly always optimal rewards a model that always gives it. In F4, the prior is drawn on both sides of one half, so that base-rate neglect overstates the posterior in some items and understates it in others, and each consecutive block of four items contains one item of each (stratum, Bayes action) cell. In F1, each block of four contains one item of each (stratum, test is the optimal first action) cell, so that half of the F1 items call for buying the test and half do not, within each stratum. This balance is what gives the value-of-information measure of Section 5.1 equal cells. If a cell cannot be filled within the generator’s draw limit, F1 is balanced on the test action alone and the trap share per test cell is reported. F2 and F3 are not rebalanced; the regret of every constant-action policy is reported beside each model’s instead (Section 5.4).

Decision-equivalent variants and the noise floor. A *decision-equivalent variant* of an item is a restatement that leaves its vector of expected losses unchanged, so that any change of action between the canonical item and the variant is an error. Three transformations are used: gain/loss framing (in F1–F3 the same outcomes are stated as amounts lost or as amounts kept; in F4 the bet is stated on H_1 or on its complement), reversal of the option order, and paraphrase, with [TODO: param: paraphrase templates] templates written before any run (the first being canonical). Equivalence is checked numerically on every item (check K3, tolerance [TODO: param: K3 tolerance]). The *flip rate* of a transformation is the share of items whose action differs between the canonical item and its variant. It is compared with the model’s *noise floor*, the rate at which the action changes when the identical prompt is submitted again (Section 4.2).

3.3. Item generation and contamination control

Item parameters are drawn from laws fixed in advance. The prior p is log-uniform on [TODO: param: prior range] (in F4, reflected about one half with equal probability); sensitivities and specificities are uniform on [TODO: param: sensitivity/specificity range] and rounded to two decimals; cost ratios (M/A , C_{FN}/C_{FP} , X/Y , price over cost) are log-uniform on [TODO: param: cost-ratio range].

and costs are integers in [TODO: param: cost range]. Each item is set in one of [TODO: param: story domains] surface domains (clinical testing, IT incidents, quality inspection and logistics), drawn uniformly. Probabilities are shown as percentages, with fixed units and currency.

Because the items are generated rather than collected, no item can have appeared in a model’s training data, and the surface stories and numbers are new. Each family has [TODO: param: items per family] items, drawn from a seed block ([TODO: param: seed block]) registered before generation and disjoint from every other seed in the project. The SHA-256 hash of each item file is committed to the public repository before the first model call, and the item files themselves are released only after all runs end, so the items cannot leak into a model or into prompt design while the study runs. The conditions run on a subset use the first [TODO: param: shared prefix per family] items of each family in its stratified order, which preserve the strata and the balance cells. Separately, a *canonical-story control* of [TODO: param: canonical-story items per family] famous textbook wordings per family is run under direct prompting only and reported apart, to show the size of any contamination effect rather than hide it.

4. Models and decision conditions

4.1. Models

The large language models (LLMs) in this study are the object of research. They are open-weight instruction models, each downloaded once at a pinned repository revision, verified by the SHA-256 hash of its weight file and served locally by an open-source inference server; no commercial API is used. Pinned weights make the tested models immutable: the results below refer to exactly these files, and anyone can rerun them. The paper states each model’s repository, revision, quantization and server version (Table 1) and claims nothing about closed-API models, other models or later versions.

The panel has [TODO: param: panel size] models, each from a different developer, chosen before any run to vary developer, architecture and size, and restricted to licences that permit research use and weights that can be downloaded without an access request: a Llama instruction model (Meta, from an ungated conversion of the released weights), a Qwen3 mixture-of-experts instruction model (Alibaba), the reasoning-native gpt-oss model (OpenAI), an OLMo instruction model (Ai2), whose training data are public, a Granite instruction model (IBM) and Phi-4-mini-instruct (Microsoft). Substitutes

Table 1: Model panel: developer, repository, revision, weight-file hash, quantization, licence and inference server, as pinned before the first model call.

[TODO: param: panel table built from the lab’s pins.json]

for each candidate were fixed in advance, for use only if a candidate could not be pinned or loaded; any substitution is recorded before that model’s first call. No model is added after the first result. With [TODO: param: panel size] models no trend with model size can be separated from developer and architecture, and none is claimed.

Decoding. Every model answers every condition with greedy decoding (temperature zero, no nucleus truncation and no repetition, frequency or presence penalty), from its own chat template, with a fixed maximum number of output tokens per condition (Appendix Appendix A). The reasoning-native model runs in its native response format at its lowest reasoning setting in every condition, including the direct one; its reasoning text is logged and never parsed. Server settings (version, quantization, context length, threads and batching) are pinned and recorded. Under greedy decoding the only remaining variation between identical requests is server nondeterminism, and the noise floor measures exactly that: [TODO: param: noise-floor items per family] items per family are resubmitted in a fresh server process with a different request order. This is the variation a user of the same deployment would see.

4.2. Conditions

All conditions share one system prompt that casts the model as an adviser to a decision-maker and asks for a single JSON object. Where the family has a probability step (F1, F2, F4), the object holds the probability of H_1 after the evidence and then the action; in F3 it holds the action only. The conditions differ in what the model is asked to produce and in who computes the decision (Table 2). Prompts, schemas and formula sentences were written before any run and were not tuned on model outputs; they are given in Appendix Appendix A.

Direct (P0). The model reads the item and answers with the JSON object only. P0 is the reference condition, and it is the condition on which the decision-equivalent variants and the canonical-story control are run.

Free reasoning (P1). The model is asked to think step by step before giving the JSON object on its last line [TODO: cite: Kojima et al. 2022 (L14), NeurIPS; bib entry pending a venues.csv row]. P1 is the strongest condition in which the model still does everything itself.

Structured direct (P2). In a single answer, the model returns its reading of the item’s quantities $\hat{q}$ in the family’s extraction schema (the same schema as the guardrail’s), then its stated probability $\hat{p}$, then its action $\hat{a}$. Because the reading, the probability and the action are all observed, P2 makes it possible to locate where a decision’s loss arises (Section 5.2). P2’s own regret is reported beside P0’s.

Program-of-thought (PoT). The model writes a Python function `decide()` that returns an action label, and the harness executes it [9] [TODO: cite: PAL (L26); bib entry pending a venues.csv row]. Execution is sandboxed: a separate process with no network or file-system access, a CPU limit of [TODO: param: PoT CPU limit], and only the standard-library modules `math`, `fractions` and `itertools` importable. PoT is run on the canonical item and on its framed variant. It is the strongest baseline in which computation is delegated to code while the model still writes the model of the problem.

Formula-given (FG). The direct condition with one added sentence per family stating the optimal decision rule in words: compare the expected loss of acting now with that of testing (F1); intervene if the posterior exceeds $C_{FP}/(C_{FP} + C_{FN})$ (F2); order the smallest quantity whose cumulative probability reaches the critical fractile, or enumerate the splits (F3); accept if the posterior times X exceeds its complement times Y (F4). FG separates not knowing the rule from failing to apply it; newsvendor biases have been reported to persist when the formula is given [TODO: cite: Liu, Chen & Zhong 2025 (L32), preprint; sole-source exception pending].

Extract-then-solve guardrail (G). The model only extracts the item’s quantities (prior, likelihoods, costs, demand table and action labels) into the family’s JSON schema, and the family’s exact solver chooses the action. G is run on the canonical item and on its framed variant, and it has its own noise floor. An oracle control runs the solver on the true quantities.

Table 2: Decision conditions. “Reads” is what the model extracts, “states” what it outputs, “decides” who selects the action.

Condition	Model outputs	Action chosen by	Also run on
P0 direct	probability ^a , action	model	framing, order, p
P1 free reasoning	reasoning, probability ^a , action	model	—
P2 structured direct	quantities, probability ^a , action	model	—
PoT program-of-thought	Python <code>decide()</code>	model-written code	framing
FG formula-given	probability ^a , action	model	—
G guardrail	quantities	exact solver	framing

^aIn F1, F2 and F4; F3 asks for the action only.

Scale. P0 and G on the canonical items are run on all [TODO: param: items per family] items of each family, because the headline measure of information pricing needs equal and large test cells in F1 (Section 5.1). Every other condition and variant is run on the first [TODO: param: shared prefix per family] items of each family, and every paired comparison uses the items the two conditions share.

4.3. Parsing, failures and checks by construction

Answers are parsed against the condition’s schema. A parse failure is retried once; whether the retry changed the output is recorded, since under greedy decoding it usually does not. An answer that still cannot be parsed, a truncated answer, or, under PoT, a syntax error, exception, timeout or invalid action label, is scored as the item’s worst action in the primary analysis and excluded in a sensitivity analysis; both are reported, with the parse-failure rate per cell. A prompt template whose parse-failure rate in the pilot ([TODO: param: pilot items per family] items per family, excluded from the main set) exceeds [TODO: param: pilot parse-failure limit] could be rewritten once, before the main run; no claim is read from the pilot.

Several properties hold by construction, and a violation is treated as a bug, not as evidence. They are checked before any outcome is read (Appendix Appendix B): the exact solver has zero regret and each heuristic behaves as its stratum requires (K1); the guardrail with oracle extraction has zero regret and no flips (K2); variants have identical expected-loss vectors (K3); mock models with fixed heuristics reproduce their analytically known regret (K4); weight-file hashes match the pinned values at the start of every job (K5); repeated runs in fresh processes agree, or their mismatch enters the

noise floor (K6); the exact posteriors reproduce a unit slope in the probability regression (K7); the F4 and F1 balance cells are filled in every block (K8, K9); and oracle P2 and PoT answers have zero regret and zero decomposition terms, while the heuristic mock under FG reproduces its known regret (K10).

5. Pre-specified analysis

The metrics, tests and reading rules of this section were fixed before any model was run and are applied as written. The study is organized around two questions. The first is whether models pay for information exactly when it is worth its cost (Section 5.1). The second is where the loss of a direct decision arises: in reading the problem, in inferring a probability from it, or in turning that probability into an action (Section 5.2). The guardrail comparisons (Section 5.3) test the design answer that follows from them. The remaining measurements (Section 5.4) replicate, on multi-action families and pinned open weights, findings that have already been reported for other tasks and panels, and they are presented as replications.

Common rules. Items are the sampling unit, and every statistic is computed per (model, family) unless stated otherwise. Confidence intervals are two-sided at the [TODO: param: confidence level of every interval, B001]% level from the item bootstrap with [TODO: param: bootstrap resamples] resamples, whose seeds come from the registered block. Differences of two proportions use the Newcombe hybrid score interval [TODO: cite: Newcombe 1998, original-source exception pending] whenever either proportion rests on fewer than [TODO: param: Newcombe event threshold] events, and always for the value-of-information measure. Comparisons repeated over families or transformations are corrected by Holm’s step-down procedure [TODO: cite: Holm 1979, original-source exception pending] at family-wise level $\alpha =$ [TODO: param: alpha]. Each claim is read per model as *supports*, *weakens*, *refutes* or *inconclusive* by the rules below, and the paper’s reading is the outcome for a majority of the panel actually run, that is, for more than half of it ([TODO: param: majority] of [TODO: param: panel size] models). Every model and family is reported, negative cells beside positive ones. A model whose calls could not all be completed within the job’s time limit after one resubmission is reported but not counted in the majority, and a claim is inconclusive if the parse-failure rate exceeds [TODO: param: parse-failure inconclusive rate] in more than half of its (model, family) cells.

5.1. Value-of-information discrimination

In F1, buying the test is the Bayes first action on half of the items of each stratum and a wrong action on the other half (Section 3.2). A decision-maker who prices information correctly buys the test on the first half and never on the second. The *value-of-information discrimination* of a model in a condition is

$$D = \Pr(\text{buys the test} \mid \text{testing is optimal}) - \Pr(\text{buys the test} \mid \text{testing is not optimal}), \quad (2)$$

estimated by the two sample proportions on the F1 items and bounded by the Newcombe interval. The exact solver has $D = 1$; a model that buys the test at a rate unrelated to its value has $D = 0$. D measures only whether the purchase tracks the value of the information; its two components, the over-testing rate (buying when testing is not optimal) and the under-testing rate (not buying when it is), are reported separately, and the direction of any mispricing is described rather than predicted.

Reading rule (C5). C5 holds that direct decisions misprice information. It is *supported* if, for a majority of models, D under P0 has an upper [TODO: param: confidence level of] bound below [TODO: param: C5 bound for P0] while D under the guardrail has a lower bound at or above [TODO: param: C5 bound for G]; *refuted* if, for a majority, D_{P0} has a lower bound at or above [TODO: param: C5 bound for P0]; *weakened* if D_{P0} is below [TODO: param: C5 bound for P0] for a majority but D_G misses [TODO: param: C5 bound for G] for a majority, so that the guardrail does not repair the pricing; and *inconclusive* if the interval of D_{P0} contains [TODO: param: C5 bound for P0] for a majority. C5 is read only on the full F1 file ([TODO: param: items per family] items, half of them in each test cell). With fewer items the lower bound of D_G cannot reach [TODO: param: C5 bound for G] even for an error-free guardrail, so the rule could not be met; at a smaller scale C5 is recorded as inconclusive and D is reported without a reading. Reported with no rule: D under P1, P2, FG and PoT, and, in F1b, the number of tests bought minus the optimal number.

5.2. Regret decomposition

A P2 answer contains the model’s reading of the quantities $\hat{q}$, its stated probability $\hat{p}$ and its action $\hat{a}$. Let a_{read} be the exact solver’s action on the

model’s own reading $\hat{q}$, and a_{inf} the Bayes action at the stated probability $\hat{p}$ under the costs in $\hat{q}$. The regret of $\hat{a}$ then splits exactly into three terms,

$$L(\hat{a}) - L^* = \underbrace{L(a_{\text{read}}) - L^*}_{\text{reading}} + \underbrace{L(a_{\text{inf}}) - L(a_{\text{read}})}_{\text{inference}} + \underbrace{L(\hat{a}) - L(a_{\text{inf}})}_{\text{action}}, \quad (3)$$

because the sum telescopes; each term is normalized by $L_{\text{max}} - L^*$ as in (1). The reading term is the loss a perfect reasoner would incur from the model’s reading alone; the inference term is the further loss of the model’s probability, given that reading; the action term is the loss of not acting on its own probability. In F3, which has no probability step, the inference and action terms are merged. The *reading share* of a (model, family) cell is its mean reading term divided by its mean total regret, estimated with the item bootstrap of the ratio of means. An answer whose $\hat{q}$, $\hat{p}$ or $\hat{a}$ cannot be parsed is excluded from the decomposition, and the exclusion rate is reported. On F1–F2 the action term is the loss-weighted counterpart of the belief–action incoherence rate of Section 5.4, and both are reported. The decomposition extends the two-term split of regret into a belief loss and an optimization residual [TODO: verify N1] by an explicit reading stage, which is where a guardrail can fail.

Reading rule (C6). C6 holds that most of the loss is not misreading. A cell whose mean total P2 regret has an upper [TODO: param: confidence level of every interval, B001]% bound below δ has no loss to locate and is marked *not applicable*. C6 is *supported* if, for a majority of models, the reading share has an upper [TODO: param: confidence level of every interval, B001]% bound below [TODO: param: C6 share threshold] in at least three of the applicable families (at least two if only three are applicable); *refuted* if, for a majority, its lower bound is above [TODO: param: C6 share threshold] in at least two families; *weakened* if support holds in only one or two families; and *inconclusive* if more than half of the (model, family) cells are not applicable or have a reading-share interval that contains [TODO: param: C6 share threshold]. The signed inference and action terms are reported per cell.

Order sensitivity. A telescoping split depends on the order in which the stages are corrected: each later term absorbs its interaction with the earlier ones, so the primary reading share is the loss of misreading alone, with the model’s own inference and action held fixed. The primary order follows the pipeline, reading then inference then action. The action term is last in

every order, because the model’s action is observed only after its own reading and inference. As a sensitivity with no reading rule, reading and inference are also split in the reverse order and the two orders are averaged, which is the two-player Shapley split [TODO: cite: Shapley 1953, original-source exception pending] of their sum. With a_{alt} the Bayes action at $\hat{p}$ under the item’s *true* costs, the reverse order assigns $L(a_{\text{alt}}) - L^*$ to inference and $L(a_{\text{inf}}) - L(a_{\text{alt}})$ to reading. Because $\hat{p}$ already embeds the model’s reading of the prior and the likelihoods, the reverse order isolates misread costs only, and the paper states this limit with the result. The Shapley reading share and its interval are reported for F1, F2 and F4; F3 has no probability and admits no second order. If the Shapley share falls on the other side of [TODO: param: C6 share threshold] for a majority of models, this is stated beside C6’s outcome, which it does not change.

5.3. Guardrail comparisons

Guardrail against the model deciding (C4). Three parts are tested for each model: (i) the guardrail’s mean regret is below P0’s in all four families and below P1’s in at least three, by the paired item bootstrap of the difference with Holm correction over families; (ii) the guardrail’s flip rate under framing minus its own noise floor has an upper [TODO: param: confidence level of every interval, B001]% bound at or below [TODO: param: C4 flip margin]; and (iii) its field-level extraction accuracy, with numbers correct within a relative tolerance of [TODO: param: extraction tolerance] and categorical fields exact, has a lower [TODO: param: confidence level of every interval, B001]% bound at or above [TODO: param: C4 extraction accuracy] in every family. C4 is *supported* if all three hold for a majority of models; *refuted* if, for a majority, the guardrail’s point estimate of regret is not below P0’s in at least three families; *weakened* if (i) holds against P0 but not against P1 in at least two families, or if (ii) or (iii) fails for a majority in any family, in which case the paper reports where extraction fails; and *inconclusive* if the paired intervals contain zero in more than half of the (model, family) cells. The item-level exact-extraction rate and the cost per decision in tokens and server seconds are reported beside it.

Verified solver against model-written code (C4b). The guardrail and PoT are compared on the same items by the paired item bootstrap of $\text{regret}(\text{G}) - \text{regret}(\text{PoT})$, with Holm correction over the four families at level α , and by the difference of their flip rates under framing, bounded by the paired interval

(Newcombe when either count is below [TODO: param: Newcombe event threshold]). C4b is *supported* if, for a majority of models, the guardrail’s regret is below PoT’s in at least three families and the flip-rate difference has an upper [TODO: param: confidence level of every interval, B001]% bound below zero; *refuted* if, for a majority, the guardrail’s point estimate of regret is at least PoT’s in at least three families; *weakened* if only one of the two parts holds for a majority; and *inconclusive* if the paired intervals contain zero in more than half of the (model, family) cells. If PoT matches the guardrail, the recommendation becomes to hand the computation to code, and the schema guardrail is one instance of it.

5.4. Replications and secondary metrics

The following measurements replicate effects already reported for direct LLM decisions and stated probabilities [3, 4, 5, 6]. Two of them repeat the closest prior designs on a new object. C1 (regret against an exact optimum) replicates the regret evaluation of Sun et al. [TODO: verify N1], which uses binary actions, on multi-action families with a costly-information option. C2(b) (belief–action incoherence) replicates the coherence finding of Yamin et al. [TODO: cite: Yamin et al. 2026 (L28), ICLR; bib entry pending a venues.csv row] with known costs, so that incoherence can be scored in expected loss.

Regret and traps (C1, C1’). C1 is *supported* if, under P0, the lower [TODO: param: confidence level] bound of mean normalized regret exceeds δ in at least three of the four families and the trap-minus-non-trap regret difference, pooled over families, has a lower bound above zero, both for a majority of models. It is *refuted* if, for a majority, the upper bound is below δ in at least three families, *weakened* if the first part holds in only one or two families or the trap difference is not above zero, and *inconclusive* if the interval contains δ in more than half of the (model, family) cells. C1’ asks whether wrong answers on trap items follow the named heuristic. In F1 and F3, where items have three or more actions, each wrong P0 answer on a trap item is compared with the share expected if wrong answers were spread uniformly, $1/(k - 1)$ for an item with k actions; the statistic is the observed heuristic share minus the mean expected share, pooled over F1 and F3. C1’ is *supported* if its lower bound is above zero for a majority, *refuted* if its upper bound is at or below zero for a majority, *weakened* if it holds in only one of the two families, and *inconclusive* for a model with fewer than [TODO: param: C1-prime minimum wrong answers]

wrong trap answers (and overall if that is true of a majority). Because a uniform spread is a weak baseline, two further expected shares are reported with no rule: that of the least-regret wrong action, and, on ordered action sets, that of the wrong action adjacent to the Bayes action.

Stated probabilities (C2). For C2(a), the stated F4 posteriors, clipped to [TODO: param: probability clip range] so that the logit is finite, are regressed as $\text{logit } \hat{p} = \beta_0 + \beta_1 \text{logit}(\text{prior}) + \beta_2 \log \text{LR} + \varepsilon$, where Bayes' rule gives $\beta_1 = \beta_2 = 1$. The fit uses only the items whose exact posterior lies in [TODO: param: C2(a) fitting window], a set defined by the item alone; on it the exact posteriors are never clipped and reproduce the unit slopes (check K7), whereas including extreme posteriors would bias the slope of an exactly Bayesian answer below one. The fit on all items and the paired difference between the model's slope and the exact posteriors' slope are reported as sensitivities. For C2(b), the *incoherence rate* in F1–F2 is the share of items on which the model's action differs from the Bayes action at its own stated probability under the item's true costs. C2 is *supported* if, for a majority of models, (a) β_1 's interval lies entirely below one and the mean absolute error $|\hat{p} - p^*|$ has a lower bound above [TODO: param: C2(a) MAE threshold], and (b) the incoherence rate has a lower bound at or above [TODO: param: C2(b) incoherence threshold]; *weakened* if only one of (a) and (b) holds; *refuted* if, for a majority, β_1 's interval contains one, the error's upper bound is below [TODO: param: C2(a) MAE threshold] and the incoherence rate's upper bound is below [TODO: param: C2(b) refutation bound]; and *inconclusive* if, for a majority, the incoherence interval contains [TODO: param: C2(b) incoherence threshold] or β_1 's interval is wider than [TODO: param: C2(a) CI-width limit]. The Brier score and the expected calibration error ([TODO: param: ECE bins] equal-mass bins, against outcomes drawn from the exact posterior with a registered seed) are reported without thresholds.

Invariance (C3). For each transformation T (framing, option order, paraphrase), the flip rate f_T under P0 is compared with the noise floor f_0 . C3 is *supported* if, for a majority of models, some transformation has an excess $f_T - f_0$ of at least [TODO: param: C3 excess] with a one-sided lower [TODO: param: confidence level of every interval, B001]% bound above zero, with Holm correction over the three transformations; *refuted* if every transformation has an upper bound of the excess below [TODO: param: C3 excess]; *weakened* if the excess holds only for option order, a known format effect [TODO: cite: Pezeshkpour & Hruschka 2024 (L11), Findings of NAACL;

`bib entry pending a venues.csv row`]; and *inconclusive* if, for a majority, the transformation with the largest excess has a point excess of at least `[TODO: param: C3 excess]` but a lower bound not above zero, or if the intervals neither exclude `[TODO: param: C3 excess]` nor lie below it. Because greedy decoding leaves a floor near zero, the difference uses the Newcombe interval whenever either rate rests on fewer than `[TODO: param: Newcombe event threshold]` events. C3 also serves as the invariance test for the guardrail and for PoT above.

Baselines and secondary metrics. Every table reports, beside the models, the exact solver, each trap heuristic, a uniformly random action and every constant-action policy (for example, always testing in F1 or always declining in F4), together with each model’s share of each action, so that a model’s regret can be read against the best policy that ignores the item. Accuracy (the share of optimal actions), the parse-failure rate, and tokens and server seconds per decision are reported per cell.

6. Results

[Intent: Report each claim’s outcome as recorded, headline first: VOI pricing (C5), then the regret decomposition (C6), then the guardrail against direct, reasoning and program-of-thought (C4, C4b), then the replications (C1, C1’, C2, C3); per model and pooled.]

6.1. *Do models price information? (C5)*

6.2. *Where the loss arises: reading, inference or action (C6)*

6.3. *Extract-then-solve against direct, reasoning and program-of-thought (C4, C4b)*

6.4. *Replications: regret, traps, probabilities and invariance (C1, C1’, C2, C3)*

7. Discussion

[Intent: Implications for designing decision support around LLMs (what to hand to the model, what to hand to a solver); limitations (open-weight panel only, synthetic items, English prompts, decomposition order); threats to validity; how the framing changes under each recorded outcome, as stated in ARGUMENT.md.]

7.1. Implications for decision support design

7.2. Limitations and threats to validity

8. Conclusion

[Intent: Restate the question, the recorded outcomes of C5 and C6, the guardrail result, and what a practitioner should do.]

Data and code availability

[Intent: Generators, solvers, prompt templates, harness, item files (released after all runs), raw model responses, pinned model identifiers and result files, deposited on acceptance with a persistent identifier.]

CRedit authorship contribution statement

Chiheb Mejrissi: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

[Intent: Owner’s statement on AI-assisted writing and coding, in Elsevier’s template wording.]

Appendix A. Prompt templates, schemas and reference solvers

[Intent: Give the fixed prompt templates per family and condition, the guardrail and P2 answer schemas, the formula-given sentences and the reference `decide()` functions.]

Appendix B. Checks by construction and generator diagnostics

[Intent: Report checks K1–K10 and the generator diagnostics (separation, strata and F1 test-action balance).]

References

- [1] M. Binz, E. Schulz, Using cognitive psychology to understand GPT-3, *Proceedings of the National Academy of Sciences* 120 (6) (2023) e2218523120. doi:10.1073/pnas.2218523120.
- [2] T. Hagendorff, S. Fabi, M. Kosinski, Human-like intuitive behavior and reasoning biases emerged in large language models but disappeared in ChatGPT, *Nature Computational Science* 3 (10) (2023) 833–838. doi:10.1038/s43588-023-00527-x.
- [3] O. Macmillan-Scott, M. Musolesi, (ir)rationality and cognitive biases in large language models, *Royal Society Open Science* 11 (6) (2024) 240255. doi:10.1098/rsos.240255.
- [4] G. Suri, L. R. Slater, A. Ziaee, M. Nguyen, Do large language models show decision heuristics similar to humans? A case study using GPT-3.5, *Journal of Experimental Psychology: General* 153 (4) (2024) 1066–1075. doi:10.1037/xge0001547.
- [5] Y. Chen, S. N. Kirshner, A. Ovchinnikov, M. Andiappan, T. Jenkin, A manager and an AI walk into a bar: Does ChatGPT make biased decisions like we do?, *Manufacturing & Service Operations Management* 27 (2) (2025) 354–368. doi:10.1287/msom.2023.0279.
- [6] L. Qiu, F. Sha, K. Allen, Y. Kim, T. Linzen, S. van Steenkiste, Bayesian teaching enables probabilistic reasoning in large language models, *Nature Communications* 17 (2026) 1238. doi:10.1038/s41467-025-67998-6.

- [7] V. Cheung, M. Maier, F. Lieder, Large language models show amplified cognitive biases in moral decision-making, *Proceedings of the National Academy of Sciences* 122 (25) (2025) e2412015122. doi:10.1073/pnas.2412015122.
- [8] H. Raiffa, R. Schlaifer, *Applied Statistical Decision Theory*, Studies in Managerial Economics, Division of Research, Graduate School of Business Administration, Harvard University, Boston, 1961.
- [9] W. Chen, X. Ma, X. Wang, W. W. Cohen, Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, *Transactions on Machine Learning Research* ArXiv:2211.12588 (2023).
URL <https://arxiv.org/abs/2211.12588>
- [10] Y. Chen, T. X. Liu, Y. Shan, S. Zhong, The emergence of economic rationality of GPT, *Proceedings of the National Academy of Sciences* 120 (51) (2023) e2316205120. doi:10.1073/pnas.2316205120.
- [11] S. G. Pauker, J. P. Kassirer, The threshold approach to clinical decision making, *New England Journal of Medicine* 302 (20) (1980) 1109–1117. doi:10.1056/NEJM198005153022003.
- [12] M. Steyvers, H. Tejada, A. Kumar, C. Belem, S. Karny, X. Hu, L. W. Mayer, P. Smyth, What large language models know and what people think they know, *Nature Machine Intelligence* 7 (2) (2025) 221–231. doi:10.1038/s42256-024-00976-7.
- [13] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, J.-R. Wen, Tool learning with large language models: A survey, *Frontiers of Computer Science* 19 (8) (2025) 198343. doi:10.1007/s11704-024-40678-2.
- [14] C. Huang, Z. Tang, S. Hu, R. Jiang, X. Zheng, D. Ge, B. Wang, Z. Wang, ORLM: A customizable framework in training large models for automated optimization modeling, *Operations Research* 73 (6) (2025) 2986–3009. doi:10.1287/opre.2024.1233.